

AI-Scout: Sistema Ibrido per il Management e la Composizione di un Team di Fantacalcio

Salatino Donato
796176
AA2025/2026

Repository Github

<https://github.com/DennySal/Icon-2025-2026.git>

Capitolo 1) Introduzione e Acquisizione della Conoscenza

1.1 Analisi del Dominio e Architettura del Sistema

La composizione di una rosa per il Fantacalcio o per il management sportivo rappresenta un classico problema di ottimizzazione sotto incertezza. I decisori devono bilanciare un budget rigoroso (crediti) per acquistare un numero predefinito di giocatori divisi per ruolo, cercando di massimizzare il rendimento atteso e minimizzare i rischi legati a infortuni, turnover o squalifiche.

Per risolvere questa sfida, il presente progetto illustra la progettazione e lo sviluppo di un Sistema Intelligente Basato su Conoscenza (KBS), concepito come un *Decision Support System* (DSS). Conformemente all'architettura teorica dei KBS (Capitolo 1 delle dispense), il sistema opera su due macro-livelli:

1. Fase Offline: Acquisizione dei dati, arricchimento semantico e apprendimento dei modelli predittivi (*Machine Learning*).
2. Fase Online: Risoluzione dei vincoli per la generazione del team ottimale (CSP) e valutazione probabilistica dei rischi ambientali (*Reti Bayesiane*).

1.2 Mappatura degli argomenti di interesse

Il sistema non si limita all'applicazione isolata di singoli algoritmi, ma integra in una pipeline coerente diversi paradigmi fondamentali dell'Ingegneria della Conoscenza studiati durante il corso. Nello specifico, il progetto copre i seguenti argomenti:

- Knowledge Graph e Ontologie (Cap. 6): Interrogazione del Web Semantico (endpoint *DBpedia*) tramite linguaggio SPARQL. Gestione dell'incompletezza dei dati (*Open World Assumption*) mediante implementazione di una logica di *Fallback Strategy*.
- Apprendimento Supervisionato (Cap. 7): Addestramento di modelli *Ensemble* (Random Forest) per task di Classificazione (Scouting predittivo). Ottimizzazione degli iperparametri tramite *GridSearchCV*, validazione rigorosa (*K-Fold Cross-Validation*) e prevenzione del sovradattamento (*Overfitting*).
- Ragionamento con Vincoli (Cap. 3): Modellazione di un *Constraint Satisfaction Problem* (CSP) per lo scheduling della squadra perfetta. Applicazione di vincoli rigidi (Budget, Ruoli) e risoluzione dello spazio degli stati tramite algoritmi di ricerca sistematica (es. Backtracking / DFS).
- Modelli di Conoscenza Incerta (Cap. 9): Costruzione di un Grafo Aciclico Diretto (DAG) per una Rete Bayesiana atta all'analisi dei rischi esogeni (es. infortuni a catena o turnover delle coppe europee).

1.3 Requisiti Funzionali e Strumenti Utilizzati

Il sistema è stato interamente sviluppato in Python (versione 3.10+). Al fine di mantenere il codice pulito e modulare, ci si è affidati a librerie di livello industriale per i task ad alto costo computazionale:

- **SPARQLWrapper**: per stabilire la connessione HTTP con DBpedia e processare i risultati.
- **pandas e numpy**: per il Data Cleaning e il Feature Engineering.
- **scikit-learn**: spina dorsale del modulo di Machine Learning (Pipeline, RandomForestClassifier, GridSearchCV).
- **imbalanced-learn**: per l'applicazione dell'algoritmo SMOTE.
- **python-constraint**: per la formulazione dichiarativa del CSP e la sua risoluzione.
- **pgmpy**: per la modellazione del DAG e l'inferenza nella Rete Bayesiana.

Struttura del Progetto: Il progetto è organizzato in modo modulare:

- **/data**: contiene i dataset grezzi e il dataset arricchito semanticamente.
- **/src**: contiene gli script Python separati per l'Estrazione Dati, il Machine Learning, il CSP e le Reti Bayesiane.
- **/models**: cartella in cui vengono salvati i modelli di ML serializzati (es. **.pkl**) per un utilizzo immediato senza ri-addestramento.

1.4 Il Dataset di Base e la Data Sparsity

Come base di partenza, si è scelto di utilizzare un dataset contenente le statistiche storiche dei giocatori dei principali campionati (es. ruolo, presenze, gol, assist, cartellini, media voto). Tuttavia, queste sole feature puramente numeriche risultano semanticamente povere. Ad esempio, per un modello predittivo, il nome di un giocatore neo-promosso o di un nuovo acquisto dall'estero è solo una stringa di testo; senza un contesto sul suo *pedigree* internazionale, l'algoritmo non ha modo di misurarne la caratura, rischiando di generare predizioni miopi. Si è reso quindi necessario introdurre della *Background Knowledge* (BK).

1.5 Integrazione con il Web Semantico (DBpedia)

Per quantificare il reale "prestigio" di ogni calciatore, si è implementata un'architettura di *Data Integration*, arricchendo il database relazionale locale con il Knowledge Graph globale di DBpedia. L'obiettivo è stato estrarre informazioni chiave, come il numero di presenze nella Nazionale maggiore o i trofei internazionali vinti in carriera. L'interazione è avvenuta tramite endpoint SPARQL (mediante la libreria Python **SPARQLWrapper**).

La "Fallback Strategy" e l'Open World Assumption Interrogando il Web Semantico ci si scontra fisiologicamente con la Open World Assumption. Ad esempio, per aggirare l'assenza di dati strutturati, l'estrazione SPARQL è stata progettata con l'operatore logico **OPTIONAL** per tre feature fondamentali:

1. **Età:** Calcolata dinamicamente estraendo la data di nascita (`dbo:birthDate`).
2. **Nazionalità (Straniero/Italiano):** Estratta tramite la proprietà `dbo:nationality`, essenziale per valutare i tempi di adattamento dei nuovi acquisti in Serie A.
3. **Trofei Vinti:** Estratti contando le istanze della proprietà `dbo:award`. In caso di assenza del dato, il sistema applica una logica di Fallback assegnando un valore di default pari a 0, evitando il crash dell'applicativo (Data Sparsity).

I dati estratti sono stati aggregati al dataset originale e sottoposti a *Data Pre-processing* (tramite `SimpleImputer` e `StandardScaler`), ottenendo una Base di Conoscenza arricchita e pronta per alimentare il modulo di Apprendimento Supervisionato

Capitolo 2) Apprendimento Supervisionato (Scouting Predittivo)

2.1 Definizione del Task e Scelta del Modello

Consolidata la Base di Conoscenza (KB) arricchita nel capitolo precedente, l'obiettivo è transitare da un approccio puramente descrittivo a uno predittivo. Come illustrato nella teoria dell'Ingegneria della Conoscenza, l'apprendimento supervisionato permette a un agente di imparare una relazione tra input e output basandosi su dati etichettati, al fine di generalizzare su nuove istanze.

Per il nostro *Decision Support System*, è stato implementato un task di **Classificazione Multiclasse**. L'obiettivo è predire la fascia di rendimento futuro di un calciatore assegnandolo a una di tre categorie:

- **Flop**: Giocatori con rendimento insufficiente o alto rischio di panchina/infortunio.
- **Titolare**: Giocatori affidabili che garantiscono presenze costanti.
- **Top Player**: Fuoriclasse in grado di portare bonus (gol/assist) decisivi.

Per questo task, si è scelto di adottare un modello appartenente alla famiglia degli *Ensemble Methods*, nello specifico il **Random Forest Classifier**. Questa scelta è motivata dalla naturale robustezza di questo algoritmo alla varianza (grazie alla tecnica del *Bagging*) e dalla sua capacità di catturare relazioni non lineari tra le feature statistiche e quelle semantiche estratte dal Web.

2.2 Gestione dello Sbilanciamento: L'algoritmo SMOTE

Analizzando la distribuzione della variabile target, è emerso un forte sbilanciamento delle classi (*Class Imbalance*). Nel dominio calcistico reale, la percentuale di "Top Player" è fisiologicamente una netta minoranza rispetto ai giocatori medi ("Titolare") o di bassa fascia ("Flop").

Addestrare il modello su questi dati avrebbe indotto il classificatore a favorire statisticamente la classe di maggioranza, ignorando i fuoriclasse.

Per risolvere il problema si è adottato l'algoritmo **SMOTE** (*Synthetic Minority Over-sampling Technique*), che genera sinteticamente nuove istanze per le classi in minoranza interpolando nello spazio delle feature tra i k-vicini (k-NN) degli esempi reali. **Scelta Architetture**
Rigorosa: Per evitare il fenomeno del *Data Leakage* (inquinamento del test set con informazioni sintetiche), lo SMOTE non è stato applicato all'intero dataset a priori, ma è stato incapsulato in un costrutto *Pipeline* della libreria *imblearn*. Questo garantisce che la generazione di dati sintetici avvenga esclusivamente sui fold di *training* durante la validazione incrociata.

Risoluzione delle Anomalie in Cross-Validation: Durante la fase di testing con 10-Fold Cross-Validation, l'algoritmo SMOTE andava in errore sui fold più piccoli a causa della mancanza di un numero sufficiente di vicini per la classe minoritaria dei "Top Player". Il problema è stato brillantemente risolto a livello ingegneristico forzando l'iperparametro

`k_neighbors=1`, garantendo la generazione sintetica robusta anche negli scenari di campionamento più estremi.

2.3 Tuning degli Iperparametri (GridSearchCV)

Per impedire al modello Random Forest di espandere i propri alberi fino a memorizzare l'intero dataset (sovradattamento), si è proceduto all'ottimizzazione degli iperparametri. È stata configurata una `GridSearchCV`. La metrica di ottimizzazione scelta per orientare la ricerca è stata l'*F1-Macro*, che calcola la media armonica tra Precision e Recall, penalizzando i modelli incapaci di riconoscere le classi minoritarie.

2.4 Valutazione delle Prestazioni (K-Fold Cross Validation)

Conformemente alle linee guida del progetto, si è evitata la valutazione basata su un singolo run. Per ottenere una stima statisticamente robusta delle capacità di generalizzazione del sistema, il modello ottimizzato è stato valutato mediante 10-Fold Cross-Validation. I risultati reali, mediati sulle 10 iterazioni, sono riassunti nella seguente tabella, evidenziando le deviazioni standard per dimostrare l'eccezionale stabilità del predittore:

Metrica	Valore Medio	Deviazione Standard
Accuracy	0.973	±0.044
Precision (Macro)	0.979	±0.034
Recall (Macro)	0.973	±0.044
F1-Score (Macro)	0.972	±0.046

Il bilanciamento pressoché perfetto tra *Precision* e *Recall* (entrambi oltre il 97%) dimostra in modo empirico l'efficacia della pipeline SMOTE: il modello non si sbilancia verso una singola classe, ma impara a riconoscere i "Top Player" con la stessa assoluta accuratezza dei "Flop".

2.5 Analisi dell'Overfitting (Learning Curves) e Feature Importance

Per dimostrare visivamente e rigorosamente l'assenza di sovradattamento (overfitting), tipico sintomo di un alto bias di varianza, sono state generate le Curve di Apprendimento (Learning Curves). Il grafico dell'andamento mostra che all'aumentare dei dati di training, l'errore di addestramento si stabilizza e l'errore di validazione decresce in modo monotono. Il progressivo restringimento del divario tra le due curve conferma che il modello possiede una sana capacità di generalizzazione.

Infine, per garantire la trasparenza del modello (Explainability), si è calcolata la Feature Importance sfruttando l'indice di Impurità di Gini dei nodi della Random Forest. L'analisi ha rivelato che, sebbene le statistiche dell'anno precedente (es. Media Voto) siano i predittori primari, le feature estratte dinamicamente dal Knowledge Graph (Età, Nazionalità e Trofei vinti) contribuiscono in modo determinante al potere predittivo globale. Questa evidenza empirica certifica il successo della metodologia ibrida proposta.

Capitolo 3) Ragionamento con Vincoli (CSP) e Composizione del Roster

3.1 Modellazione Formale del Problema

Nel dominio fantacalcistico, la fase dell'asta non avviene valutando un giocatore alla volta in modo isolato, ma componendo un portafoglio (la rosa) che deve rispettare rigide regole matematiche e tattiche. Come descritto nel materiale didattico, questo scenario può essere formalizzato a tutti gli effetti come un Problema di Soddisfacimento di Vincoli (CSP), il quale è definito da un insieme di variabili, ognuna con un proprio dominio, e da un insieme di vincoli.

- **Variabili:** I 25 "slot" della rosa da riempire, suddivisi in 3 Portieri (P_1, P_2, P_3), 8 Difensori ($D_1 \dots D_8$), 8 Centrocampisti ($C_1 \dots C_8$) e 6 Attaccanti ($A_1 \dots A_6$).
- **Domini:** Il database dei giocatori disponibili nel listone, precedentemente analizzati e classificati dal nostro modulo di Machine Learning.

3.2 Definizione degli Hard e Soft Constraints

Per garantire la validità formale della rosa, il sistema deve filtrare le assegnazioni attraverso una serie di vincoli rigidi (hard constraint) che specificano le combinazioni lecite di assegnazioni.

- **AllDifferent Constraint:** Un giocatore non può essere assegnato a più di una variabile contemporaneamente (non si può comprare due volte lo stesso calciatore).
- **Vincolo Finanziario (Budget):** La somma del costo in crediti di tutti e 25 i giocatori non deve superare il tetto massimo di 500 crediti.
- **Vincoli di Ruolo:** I giocatori scelti per le variabili P_i devono appartenere al dominio dei portieri, i D_i ai difensori, e così via.

Oltre a questi, introduciamo dei **vincoli flessibili (soft constraint)** per codificare le preferenze fra le diverse assegnazioni e massimizzare la qualità della squadra. Sfruttando le predizioni del Capitolo 2, imponiamo che il sistema debba selezionare almeno un numero minimo di giocatori classificati come "Top Player" per ogni reparto, massimizzando il rendimento atteso. Questo trasforma il task in un vero e proprio **Problema di Ottimizzazione Vincolata**.

3.3 Risoluzione del CSP e Algoritmi di Ricerca

Lo spazio di ricerca per comporre una squadra scegliendo 25 elementi su oltre 500 disponibili in Serie A è mastodontico. L'approccio *Generate-and-Test*, che genera e controlla in modo sistematico le possibili assegnazioni totali, risulta totalmente inefficiente a causa della sua complessità esponenziale.

Per dominare la complessità, il sistema sfrutta algoritmi di ricerca su grafo supportati da tecniche di *Separazione dei Domini* e propagazione. Nello specifico, si implementa un algoritmo di ricerca sistematica in profondità (DFS) con *Backtracking*, potenziato da algoritmi basati su consistenza come la **Generalized Arc Consistency (GAC)**. La GAC permette di eliminare preventivamente i valori inconsistenti dai domini: ad esempio, se i primi 24

giocatori acquistati costano complessivamente 499 crediti, l'algoritmo rimuove istantaneamente dal dominio dell'ultimo giocatore tutti i calciatori che costano più di 1 credito, sfoltendo drasticamente i rami inutili (pruning).

3.4 Adversarial Testing e Risultati

Per testare la robustezza del motore di inferenza (implementato tramite il modulo `python-constraint`), sono stati eseguiti diversi stress test pratici:

- **1. Tentativo di violazione (Stress Test):** Fornendo al sistema un set di parametri volutamente inconsistenti (es. imponendo l'acquisto di 5 Top Player ma riducendo il budget totale a soli 200 crediti), l'algoritmo esaurisce lo spazio di ricerca e rigetta la configurazione.

```
=====
FASE 3: RISOLUZIONE VINCOLI (CSP) - FANTACALCIO
=====
Vincoli attivi: Max 200 Crediti (Asta ridotta per test), Schema (3P, 8D, 8C, 6A).
Vincolo Ibrido (ML): Minimo 5 Top Player in rosa.
... Pruning dei domini in corso (GAC) ...
... Ricerca DFS con Backtracking ...

Trovate 0 configurazioni valide in 0.562 secondi.
```

- **2. Risoluzione Ottimale:** Ripristinando il vincolo finanziario reale (500 crediti) e lasciando il sistema libero di agire sui domini, l'algoritmo di propagazione converge con un'efficienza notevole, analizzando centinaia di migliaia di combinazioni e restituendo un roster valido che massimizza la presenza di talenti.

```
Trovate 226764 configurazioni valide in 2.043 secondi.

ROSA APPROVATA (Miglior compromesso ML/Budget)
=====
Budget Consumato: 338 / 500 crediti
Top Player (ML) inseriti: 5
[PORTIERI]: Sommer (15cr), Skorupski (1cr), Mirante (1cr)
[DIFENSORI]: Bremer (18cr), Pirola (4cr), Darmian (12cr), Danilo (10cr), Gatti (14cr), Perez (3cr), Baschirotto (5cr), Bisseck (6cr)
[CENTROCAMPISTI]: Calhanoglu (20cr), Frendrup (15cr), Ederson (18cr), Cristante (12cr), Mkhitaryan (19cr), Locatelli (10cr), Bove (8cr), Mandragora (5cr)
[ATTACCANTI]: Zirkzee (28cr), Dybala (35cr), Thuram (30cr), Gudmundsson (22cr), Pinamonti (15cr), Lucca (12cr)
=====
```

Capitolo 4) Ragionamento Probabilistico e Analisi del Rischio (Reti Bayesiane)

4.1 La Necessità di Modellare l'Incertezza

Il modulo CSP analizzato nel capitolo precedente opera in un dominio di certezza deterministica: dati i vincoli e il budget, la composizione della rosa è matematicamente valida o invalida. Tuttavia, nel calcio reale, l'esito di una partita e le prestazioni settimanali di un giocatore sono influenzati da variabili esogene, stocastiche e parzialmente inosservabili. Nel Fantacalcio, un fattore critico è rappresentato dai *Malus* (ammonizioni ed espulsioni), che possono vanificare un'ottima prestazione. Per supportare il Fanta-Manager nella scelta della formazione titolare da schierare ogni domenica, il sistema deve saper ragionare sotto incertezza, facendo ipotesi e valutandone l'attendibilità. A tale scopo, l'ultimo strato dell'architettura implementa una **Rete Bayesiana** fungendo da strumento di *Risk Analysis*.

4.2 Progettazione del Grafo Aciclico Diretto (DAG)

Conformemente alla teoria, la Rete Bayesiana (BN) è stata formalizzata tramite un Grafo Aciclico Diretto (DAG) utilizzando la classe `DiscreteBayesianNetwork` della libreria `pgmpy` (aggiornata rispetto alle versioni deprecate). Per valutare il rischio disciplinare di un giocatore, sono state individuate le seguenti variabili:

- **Importanza_Match** (stati: Normale, Derby/Scontro Diretto): I match ad alta tensione generano storicamente più falli.
- **Severità_Arbitro** (stati: Bassa, Alta): Variabile indipendente che modella la propensione dell'arbitro a estrarre cartellini.
- **Rischio_Malus** (stati: Nessuno, Giallo, Rosso): Il nodo figlio (variabile dipendente) che rappresenta l'esito finale per il giocatore.

4.3 Probabilità Condizionate (CPD e CPT)

Per ogni nodo della rete è stata definita una distribuzione di probabilità condizionata (CPD) espressa in forma di tabelle (CPT). Per i nodi radice (senza genitori, come la Severità dell'Arbitro), sono state utilizzate le probabilità a priori basate sulle statistiche reali del campionato. Per il nodo figlio `Rischio_Malus`, la CPT è stata ingegnerizzata per modellare la combinazione delle cause: se la variabile `Severità_Arbitro` assume il valore *Alta* e `Importanza_Match` è *Derby*, la probabilità condizionata di ottenere il valore *Rosso* (espulsione) per il nodo `Rischio_Malus` subirà un drastico aumento.

4.4 Inferenza Esatta e Decision Support

L'obiettivo operativo della rete è calcolare la distribuzione a posteriori del nodo di query (**Rischio_Malus**) data l'evidenza disponibile. Acquisita l'evidenza dinamica dall'utente, il sistema esegue un'Inferenza Esatta marginalizzando le variabili non osservate tramite l'algoritmo di *Variable Elimination*.

Di seguito si riportano i risultati reali dell'inferenza calcolata dal sistema per due scenari antitetici, che dimostrano le capacità di Risk Analysis della rete:

Scenario A: Match Normale e Arbitro Permissivo

- Probabilità Nessun Malus (0): **80.00%**
- Probabilità Cartellino Giallo (1): **18.00%**
- Probabilità Cartellino Rosso (2): **2.00%**

Scenario B: Derby/Scontro Diretto e Arbitro Severo

- Probabilità Nessun Malus (0): **15.00%**
- Probabilità Cartellino Giallo (1): **60.00%**
- Probabilità Cartellino Rosso (2): **25.00%**

Supporto Decisionale: Come si evince dall'output matematico, nel caso B la combinazione di fattori ambientali ostili fa volare la probabilità di subire un'espulsione dal **2% al 25%**. Il risultato dell'inferenza restituisce al decisore (il Fanta-Manager) un supporto decisionale quantitativo per scegliere se schierare un difensore a rischio oppure optare per una riserva che disputa una partita statisticamente più tranquilla.

Capitolo 5) Conclusioni e Sviluppi Futuri

5.1 Conclusioni

Il progetto *AI-Scout* dimostra concretamente come sia possibile e proficuo far cooperare paradigmi concettualmente dissimili dell'Intelligenza Artificiale all'interno di un unico *Decision Support System*. La Base di Conoscenza Semantica ha permesso di abbattere la dipendenza da feature esplicite statiche, arricchendo i dati con la *Background Knowledge* proveniente dal Web Semantico; l'Apprendimento Supervisionato ha astratto pattern latenti per la classificazione predittiva delle prestazioni; il Ragionamento con Vincoli (CSP) ha ottimizzato l'uso delle risorse economiche per la composizione della rosa; infine, il Ragionamento Bayesiano ha permesso di incapsulare e gestire i fattori stocastici ambientali legati ai malus disciplinari. L'integrazione di questi moduli conferma la necessità e l'efficacia degli approcci ibridi (Neuro-Simbolici) per lo sviluppo di Sistemi Basati su Conoscenza in domini complessi e incerti.

5.2 Sviluppi Futuri

Da un punto di vista progettuale e funzionale, il sistema potrebbe essere esteso nelle seguenti direzioni:

- **Deep Learning e Reti Neurali Sequenziali:** Implementazione di reti ricorrenti (RNN o LSTM) per analizzare le serie temporali delle prestazioni settimanali dei giocatori, catturando le "strisce positive" o i cali di forma.
- **Analisi del Sentiment (NLP):** Estrazione automatica di notizie e tweet (tramite *Infobot* o *Webbot*) per valutare il morale del giocatore o le tensioni con l'allenatore, da inserire come nuova variabile di evidenza nella Rete Bayesiana.
- **Agenti Competitivi (Multi-Agent System):** Sviluppo di agenti simulati per testare la fase di asta, modellando le strategie di rilancio degli avversari in scenari parzialmente osservabili.

Bibliografia e Sitografia

Testi di Riferimento (Ingegneria della Conoscenza):

- Poole, D., & Mackworth, A. (2023). *Artificial Intelligence: Foundations of Computational Agents* (3rd ed.). Cambridge University Press. (Riferimento primario per KBS, CSP, Machine Learning e Reti Bayesiane).
- Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: a modern approach* (4th ed.). Pearson. (Riferimento per algoritmi di ricerca e logica).
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer. (Riferimento per l'Apprendimento Supervisionato e l'Overfitting).
- Hogan, A., et al. (2021). *Knowledge graphs*. ACM Comput. Surv., 54:1–71:37. (Riferimento per il Web Semantico e le Ontologie).

Strumenti e Librerie Software:

- **Scikit-learn:** Libreria per Machine Learning in Python (Modelli Ensemble, GridSearchCV, Pipeline). (<https://scikit-learn.org>)

- **Imbalanced-learn:** Strumento per la gestione del class imbalance tramite SMOTE.
(<https://imbalanced-learn.org>)
- **Pgmpy:** Libreria Python per l'apprendimento e l'inferenza su Reti Bayesiane.
(<https://pgmpy.org>)
- **Python-constraint:** Modulo per la modellazione e risoluzione di Constraint Satisfaction Problems (CSP).
- **SPARQLWrapper:** Interfaccia Python per l'interrogazione di endpoint SPARQL (DBpedia).